PALS's LAW

A Formal Specification of LLM Output Unreliability as an Architectural Invariant

Pedro Anisio de Luna e Silva (PALS)

Document version 1.5.4
April 2026

Disclaimer

No information within this document should be taken for granted. Any statement or premise not backed by a real logical definition or verifiable reference may be invalid, erroneous, or a hallucination. Mathematical claims that cannot be attributed to a published theorem are first-principles derivations and must be treated as such.

Generated by: PALS alongside Claude Sonnet 4.6 via claude.ai. LaTeX conversion by Claude Opus 4.6. *Date:* 2026-04-03.

Status: Draft — peer review pending
Version history:

- v1.1 — Demoted unestablished strong form; elevated operative form (§3.2) as primary claim.
- v1.2 — Added working definition of "realistic distribution" (§3.2); added independence caveat (§3.4); added `MODEL_VERSION` to contract block.
- v1.3 — Restructured section order to match logical dependency graph; merged practitioner artifacts; corrected stale cross-references.
- v1.4 — Added `ERR_REASONING` to taxonomy; added prompt-injection scope note; added Sharma et al. (2023); added §7.5 (Boolean predicate disclosure); formalized Corollary 5 as labeled hypothesis.
- v1.5 — Added §1 positioning paragraph and §4.1 theoretical foundations (Kalai & Vempala, Xu et al., Karpowicz, Suzuki et al.); completed 9-class taxonomy coverage; added §7.6 (computability-theoretic vs. practical non-negligibility); added Berglund et al. and Zhou et al. (IFEval) as empirical anchors; tightened §3.2 dom(Σ) restriction and Corollary 2; added scope notes for `ERR_POLICY`, `ERR_TOOL_USE`, and verification cost model.

Contents

1 Preamble **2**

2 Informal Statement **3**

3 Formal Statement **3**
3.1 Definitions . . . 3
3.2 The Law (Operative Form) . . . 3
3.3 The Law (Existential Form) . . . 4

3.4 The Pipeline Corollary (Compounding) 4

4 Empirical Support **5**
4.1 Theoretical Foundations 6

5 Taxonomy of LLM Errors **7**

6 Argument Sketch **8**
6.1 Probabilistic generation is not deterministic truth 8
6.2 Finite parameters cannot encode unbounded world knowledge 8
6.3 Evaluation is not generation 9

7 Limitations **9**
7.1 The error predicate ε is not computable in general 9
7.2 δ is task-dependent, model-dependent, and distribution-dependent 9
7.3 Independence assumption in pipeline compounding is approximate 9
7.4 Verification coverage vs. verification depth is unresolved 9
7.5 The Boolean error predicate is a deliberate simplification 9
7.6 Computability-theoretic impossibility vs. practical non-negligibility 9

8 Architectural Corollaries **10**
8.1 Corollary 1 — Appearance of correctness is not correctness 10
8.2 Corollary 2 — Trust accumulation is prohibited 10
8.3 Corollary 3 — Verification scope must match error taxonomy 10
8.4 Corollary 4 — Silent acceptance is an architectural defect 10
8.5 Corollary 5 — Capability growth shifts the verification problem, not away from it 10

9 Practitioner Artifacts **11**
9.1 Full Contract Block 11
9.2 Short-Form (headers, PR descriptions, commit messages) 12
9.3 Inline Banner (single-line, for code comments) 12
9.4 CLAUDE.md Integration Block 12

1 Preamble

PALS's Law is an engineering principle asserting that **LLM output error is not an exceptional condition but a statistical invariant of the model class**, and that any system failing to treat it as such contains an architectural defect — regardless of how correct the output appears at inspection time.

The law is named after its author in the tradition of eponymous software engineering principles (Hyrum's Law, Postel's Law, Zawinski's Law) whose value lies not in novelty but in making an implicit, widely violated assumption *explicit and enforceable*.

The general principle of engineering around unreliable components is well established in safety-critical systems (Triple Modular Redundancy, N-version programming, IEC 61508, DO-178C, ISO 26262). The theoretical inevitability of LLM hallucination has been independently established by Kalai & Vempala (STOC 2024), who proved a statistical lower bound for calibrated language models, and by Xu, Jain & Kankanhalli (2024), who proved it via computability-theoretic diagonalization (see §4.1 for full references). PALS's Law does not claim priority over these results. Its contribution is the bridge from theoretical inevitability to a specific, named architectural prescription: any system consuming LLM output without a declared verification boundary contains a structural defect. The constituent insights are established; their packaging as an enforceable engineering contract is the contribution.

2 Informal Statement

Across any realistic deployment, LLMs reliably produce errors — at a rate that is non-zero and non-negligible. Omissions, hallucinations, partial completions, and silent failures are not edge cases — they are statistical properties of the model class. No individual call is guaranteed to fail; the *distribution* over calls guarantees failure at scale.

Failing to verify LLM output is therefore not a bug in the generated artifact. It is an **architectural omission** in the system that consumed it.

Every pipeline, agent, or workflow that accepts LLM output MUST treat that output as **untrusted, incomplete, and unverified by default.** Verification is not optional post-processing — it is a first-class design concern, on par with authentication and input validation.

Absence of a verification layer is a design defect, regardless of how correct the LLM output appears to be.

3 Formal Statement

3.1 Definitions

Let:

- $\mathcal{M}$ be the class of autoregressive transformer language models.
- $M \in \mathcal{M}$ be any concrete model with parameter set θ.
- $\mathcal{X}$ be the space of all valid input prompts.
- $\mathcal{Y}$ be the space of all possible output sequences.
- $x \in \mathcal{X}$ be any specific prompt.
- $y = M(x)$ denote one sampled output, where $y \sim P_\theta(\cdot \mid x)$.
- Σ be a ground-truth semantic specification (a partial function $\Sigma : \mathcal{X} \to \mathcal{Y}$ mapping prompts to correct outputs).
- $\varepsilon(y, x) \in \{0, 1\}$ be a Boolean error predicate: $\varepsilon(y, x) = 1$ iff y deviates from $\Sigma(x)$ in any dimension enumerated in §5.

3.2 The Law (Operative Form)

$$\forall M \in \mathcal{M},\ \forall \text{ realistic } \mathcal{D} \text{ over } \mathcal{X} : \quad \mathbb{E}_{x\sim\mathcal{D}}\big[\varepsilon(M(x), x)\big] \geq \delta > 0 \tag{1}$$

where δ is non-negligible — measurably above zero for any extant model on any realistic task distribution. The expectation is implicitly restricted to $x \in \mathrm{dom}(\Sigma)$ — inputs for which a ground-truth specification exists. When $\Sigma(x)$ is undefined (e.g., creative or subjective prompts), $\varepsilon(y, x)$ is undefined and the expectation does not apply.

Working definition — "realistic distribution": A distribution $\mathcal{D}$ over $\mathcal{X}$ is *realistic* if it has non-negligible probability mass on inputs that invoke at least one of: (a) factual claims verifiable against external ground truth, (b) instruction following with checkable constraints, or (c) structured generation with a declared schema. Distributions adversarially engineered to minimize error rates — e.g., restricted to inputs whose correct answer is trivially encoded in the model's top-1 output — are excluded. This definition is a working characterization, not a formal one; formalizing it for a specific deployment context is a precondition for quantifying δ in that context. See §7.2.

This is the operative claim. The value of δ is task- and model-dependent but is never zero, and published benchmarks place it well above machine-epsilon for all models tested to date (see §4).

3.3 The Law (Existential Form)

A weaker claim that is formally establishable without empirical support:

$$\forall M \in \mathcal{M} : \ \exists x \in \mathcal{X} \text{ such that } P_\theta(\varepsilon(M(x), x) = 1) > 0 \tag{2}$$

Reading: For every model in the class, there exists at least one input on which incorrect output has positive probability. This follows from the finite parameter argument in §6.2 and requires no empirical grounding.

> **Note on the universal form.** A version asserting $P(\varepsilon = 1) > 0$ for *all* $x \in \mathcal{X}$ would follow trivially from softmax non-degeneracy under sampled generation — any softmax output distribution with $T > 0$ assigns non-zero probability to any token, including incorrect ones. However, this reduces to "sampling is stochastic" and is too weak to establish non-negligible error rates in practice. The operative claim (§3.2) carries the engineering weight; the universal form is background motivation only.
> Standard inference strategies further reduce the universal form's practical relevance: greedy decoding (argmax) selects only the top token; top-k and top-p (nucleus) sampling explicitly truncate the distribution; temperature scaling as $T \to 0$ pushes toward a point mass. Under any of these, the non-zero tail probability for incorrect tokens is astronomically small — well below hardware-level stochasticity. This reinforces why the operative claim (§3.2), grounded in measured error rates on realistic tasks, is the one that carries engineering weight.

3.4 The Pipeline Corollary (Compounding)

For a pipeline $\mathcal{P} = (M_1, M_2, \ldots, M_n)$ where each step invokes an LLM call, if calls are treated as approximately independent:

$$P(\text{pipeline is error-free}) = \prod_{i=1}^{n} (1 - p_i) \tag{3}$$

where $p_i = P(\varepsilon(M_i(x_i), x_i) = 1) \geq \delta > 0$ for all i.

> **Independence caveat:** Real pipelines share context — outputs from step i become inputs to step $i + 1$. This produces correlated errors, and the product formula can be wrong in *both* directions: errors that cluster early may be caught by a downstream verifier (formula too pessimistic), or an early error can corrupt downstream context and cascade (formula too optimistic). The product formula is a lower-bound motivation, not a deployable risk model. See §7.3.

Therefore:

$$P(\text{at least one error in pipeline}) = 1 - \prod_{i=1}^{n} (1 - p_i) \xrightarrow{n \to \infty} 1 \tag{4}$$

Implication: Multi-step agentic pipelines without per-step verification have failure probability that approaches 1 monotonically as pipeline length grows. The limit holds because the operative form guarantees $p_i \geq \delta > 0$ for each step, which implies $\sum p_i = \infty$ and therefore $\prod(1 - p_i) \to 0$.

4 Empirical Support

The following references ground the operative form (§3.2) empirically. They establish that δ is non-negligible — not merely formally positive — across model classes and task types. Where a DOI is listed, it is provided as claimed; the reader is responsible for independent verification.

Reference	Relevance	Note
Ji, Z., et al. (2023). "Survey of Hallucination in Natural Language Generation." *ACM Computing Surveys*, 55(12). DOI: 10.1145/3571730	Establishes hallucination as a documented, persistent, cross-model phenomenon — not an artifact of any specific architecture.	High confidence.
Maynez, J., et al. (2020). "On Faithfulness and Factuality in Abstractive Summarization." *ACL 2020*. DOI: 10.18653/v1/2020.acl-main.173	Distinguishes intrinsic from extrinsic hallucination; demonstrates measurable rates in generation tasks. Pretraining reduces rates but does not eliminate them.	High confidence.
Lin, S., et al. (2022). "TruthfulQA: Measuring How Models Mimic Human Falsehoods." *ACL 2022*. arXiv:2109.07958.	Benchmark quantifying factual error rates across models; no model scores 100%. Note: TruthfulQA has known saturation issues for frontier models. Cited as historical evidence of non-zero error rates.	High confidence.
Kadavath, S., et al. (2022). "Language Models (Mostly) Know What They Know." arXiv:2207.05221. Anthropic.	Addresses the gap between internal belief and expressed confidence (`ERR_CALIBRATION`). The gap narrows with scale — but a narrowing gap is still a non-zero gap.	High confidence.
Perez, E., et al. (2022). "Discovering Language Model Behaviors with Model-Written Evaluations." arXiv:2212.09251. Anthropic.	Documents sycophancy as a measurable, reproducible behavior.	High confidence.
Sharma, M., et al. (2023). "Towards Understanding Sycophancy in Language Models." arXiv:2310.13548.	Direct mechanistic analysis of sycophancy: models systematically favor responses aligned with perceived user preferences. Confirms `ERR_SYCOPHANCY` as a distinct, reproducible failure class.	High confidence.
Berglund, L., et al. (2023). "The Reversal Curse: LLMs trained on 'A is B' fail to learn 'B is A'." arXiv:2309.12288.	Demonstrates a specific, reproducible instance of `ERR_REASONING`: models trained on directional factual associations (A→B) fail to generalize to the reverse (B→A). Confirms reasoning failure as distinct from hallucination.	High confidence.

Reference	Relevance	Note
Zhou, J., et al. (2023). "Instruction-Following Evaluation for Large Language Models." arXiv:2311.07911.	IFEval: benchmark measuring instruction-following accuracy across 25 verifiable constraint types. No model achieves 100% compliance. Directly grounds `ERR_INSTRUCTION`.	High confidence.

Caveat on references: The above citations were included based on high recall confidence. Readers building on this document should independently verify each reference before treating it as authoritative.

Coverage note: The empirical references directly support five of nine error classes: `ERR_HALLUCINATION` (Ji, Maynez, Lin), `ERR_SYCOPHANCY` (Perez, Sharma), `ERR_CALIBRATION` (Kadavath), `ERR_REASONING` (Berglund), and `ERR_INSTRUCTION` (Zhou/IFEval). The remaining four classes (`ERR_OMISSION`, `ERR_SCHEMA`, `ERR_TRUNCATION`, `ERR_SEMANTIC`) are defined in the taxonomy (§5) and are observable in practice, but are not individually grounded by a dedicated reference in this section. The operative form (§3.2) asserts non-negligible aggregate error across all classes combined.

4.1 Theoretical Foundations

The following references independently establish the theoretical inevitability of LLM error. They ground the *existential form* (§3.3) formally.

Reference	Relevance	Note
Kalai, A. T. & Vempala, S. S. (2024). "Calibrated Language Models Must Hallucinate." *STOC 2024*, pp. 160–171. DOI: 10.1145/3618260.3649777	Proves a statistical lower bound on hallucination rate for calibrated LMs, tied to the monofact rate. The strongest published formalization of hallucination inevitability.	High confidence — published at STOC.
Xu, Z., Jain, S., & Kankanhalli, M. (2024). "Hallucination is Inevitable: An Innate Limitation of Large Language Models." arXiv:2401.11817.	Uses diagonalization to prove that for any computably enumerable set of LLMs, there exists a computable ground truth on which every model must hallucinate.	High confidence.
Karpowicz, M. P. (2025). "On the Fundamental Impossibility of Hallucination Control in Large Language Models." arXiv:2506.06382.	Proves impossibility via mechanism design theory and proper scoring rules.	High confidence.

Reference	Relevance	Note
Suzuki, A., et al. (2025). "Hallucinations are inevitable but can be made statistically negligible." arXiv:2502.12187.	Proves the complementary positive result: while hallucination-triggering inputs are infinite, their probability mass under a realistic distribution can be made arbitrarily small. See §7.6 for discussion.	High confidence.

Relationship to PALS's Law: These results independently establish what PALS's Law takes as given: that LLM error is not an engineering defect to be debugged but a structural property of the model class. The law's contribution is not the theoretical result but its packaging as a named, testable, enforceable architectural contract with a concrete error taxonomy and practitioner artifacts.

5 Taxonomy of LLM Errors

PALS's Law is agnostic to the specific *kind* of error that occurs. The error predicate ε (defined in §3.1) covers the following failure modes, which are not mutually exclusive:

Class	Identifier	Definition
Hallucination	`ERR_HALLUCINATION`	Asserting a false factual claim with apparent confidence, including fabricated references, non-existent APIs, or incorrect statistics.
Omission	`ERR_OMISSION`	Silently dropping required content — instructions followed partially, constraints missed, fields absent from structured output.
Schema violation	`ERR_SCHEMA`	Output structurally non-conformant with the declared format (JSON parse failure, missing keys, wrong types).
Partial completion	`ERR_TRUNCATION`	Output cut short due to token budget, generation stopping heuristics, or streaming interruption.
Sycophantic drift	`ERR_SYCOPHANCY`	Output shaped by perceived user preference rather than truth; agreement substituting for accuracy.
Instruction failure	`ERR_INSTRUCTION`	Violation of explicit constraints stated in the prompt (language, length, format, prohibited content).
Calibration failure	`ERR_CALIBRATION`	Expressed confidence misaligned with actual reliability; under- or over-hedging.
Reasoning failure	`ERR_REASONING`	Correct facts, invalid composition — multi-step inference breakdowns, reversal failures, and logical contradictions from inability to reliably chain premises across steps. Distinct from `ERR_HALLUCINATION`: the individual facts may be correctly represented; the error is in their composition.
Semantic drift	`ERR_SEMANTIC`	Correct surface form, wrong meaning — paraphrase that inverts, weakens, or subtly misrepresents the intended claim.

Each class requires a distinct detection strategy. **No single verifier can detect all classes.** This is the foundational motivation for treating verification as a first-class architectural concern rather than a single post-processing step.

Scope note — adversarial contexts: The classes above cover *intrinsic* model failures. In adversarial contexts (especially agentic deployments with tool access), prompt injection produces $\varepsilon = 1$ outputs by exploiting the model's instruction-following behavior via untrusted content in its context. This is an *extrinsic* threat and is out of scope for this intrinsic error taxonomy but requires separate threat-model analysis in any agentic deployment.

Scope note — policy and compliance violations: Enterprise deployments commonly encounter outputs that are factually correct yet violate business rules or regulatory constraints. A hypothetical `ERR_POLICY` class would capture this — but it is *extrinsic* to the model's semantic correctness: $\varepsilon(y, x) = 0$ while a separate policy predicate $\pi(y) = 1$. This taxonomy restricts itself to *intrinsic* model errors.

Scope note — multimodal and agentic error classes: The formalization assumes $\mathcal{X}$ is a space of text prompts and $\mathcal{Y}$ is a space of text output sequences. As LLMs increasingly process images, audio, and structured tool calls, the taxonomy may require extension — notably an `ERR_TOOL_USE` class for agentic deployments. The verification cost model is unaddressed: for some error classes, verification cost may exceed generation cost. These are acknowledged as out of scope for v1.5.

6 Argument Sketch

The following arguments support the existential form (§3.3) and motivate the operative form (§3.2). They are informal arguments, not formal proofs.

6.1 Probabilistic generation is not deterministic truth

[Informal argument — empirical grounding in §4.]

All $M \in \mathcal{M}$ generate tokens by sampling from a learned conditional distribution:

$$P_\theta(y \mid x) = \prod_{t=1}^{|y|} P_\theta(y_t \mid y_{<t},\, x) \tag{5}$$

No learned distribution over a discrete vocabulary exactly matches any ground-truth semantic specification Σ on all inputs. Were it to do so, the model would constitute a solution to arbitrary natural language understanding — a problem with no known finite-parameter solution for the full space $\mathcal{X}$.

This establishes: $\exists\, x$ such that $P_\theta(\varepsilon = 1) > 0$.

6.2 Finite parameters cannot encode unbounded world knowledge

[Informal argument. The cardinality claim uses pigeonhole reasoning; Xu, Jain & Kankanhalli (2024), cited in §4.1, provide the rigorous version via computability-theoretic diagonalization.]

The parameter count $|\theta|$ is finite. The set of true factual propositions over which Σ is defined is effectively unbounded (and dynamic). The number of distinct representable belief states in θ is bounded by the model's capacity. Given that the set of true propositions is unbounded, there

exist true propositions that are either unrepresented or misrepresented in θ. On inputs that invoke those propositions, $P(\varepsilon = 1) > 0$ by construction.

6.3 Evaluation is not generation

[Informal argument — empirical grounding in Kadavath et al. (2022), cited in §4.]

Even if M has encoded a correct belief about $\Sigma(x)$, the sampling process that produces y may not faithfully surface that belief. This is precisely the calibration failure class (`ERR_CALIBRATION`).

7 Limitations

The following are honest limitations of the current formalization. They scope the law's precision without invalidating it.

7.1 The error predicate ε is not computable in general

For `ERR_HALLUCINATION`, determining whether a claim is false requires access to ground truth, which is not always available. The law asserts the *existence* of errors, not a general algorithm for finding them.

7.2 δ is task-dependent, model-dependent, and distribution-dependent

The operative form (§3.2) asserts non-negligibility but does not provide a universal bound. Calibrating δ for a specific deployment requires: (a) a concrete characterization of the task distribution, and (b) empirical measurement on that distribution for the target model.

7.3 Independence assumption in pipeline compounding is approximate

In practice, LLM calls in a pipeline share context: errors in early steps propagate as corrupted inputs to later steps. The product formula in §3.4 can be wrong in both directions. It motivates the architectural consequence; it is not a deployable risk quantification without a correlation model.

7.4 Verification coverage vs. verification depth is unresolved

The contract checklist (§9.1) identifies *which error classes* a verifier covers, but does not specify the detection power within each class.

7.5 The Boolean error predicate is a deliberate simplification

The predicate $\varepsilon(y, x) \in \{0, 1\}$ treats all errors as equivalent regardless of severity. The binary choice is defensible for the law's primary purpose: establishing that verification layers are mandatory. For quantifying δ or designing severity-weighted verification systems, a graded predicate $\varepsilon(y, x) \in [0, 1]$ is the appropriate extension.

7.6 Computability-theoretic impossibility vs. practical non-negligibility

The theoretical foundations cited in §4.1 establish that the *set* of inputs on which any LLM must err is infinite. Suzuki et al. (2025) prove the complementary result: the *probability mass* on those inputs can be made arbitrarily small with sufficient training data quality and quantity. These results are not contradictory — they operate at different levels of analysis.

The operative form (§3.2) is an *empirical* claim: δ is non-negligible for all extant models on all realistic distributions. If future training regimes were to drive δ below any measurement

threshold for a specific restricted domain, the operative form would not apply to that domain — but the existential form (§3.3) would still hold, and the architectural prescription would remain justified as a conservative design principle.

The practical relevance of Suzuki et al.'s result depends on the closed-world assumption: the training distribution must adequately represent the deployment distribution. Under the open-world assumption, the probability mass on failure inputs cannot be driven arbitrarily low. Real deployments typically operate under mixed conditions.

8 Architectural Corollaries

The following corollaries are direct implications of the law and its argument sketch. They are not recommendations — they are logical consequences.

8.1 Corollary 1 — Appearance of correctness is not correctness

A system that validates LLM output by inspection on a finite test set has not demonstrated error-freedom. It has demonstrated error-absence on the tested inputs.

Practical implication: Manual review during development does not substitute for runtime verification in production.

8.2 Corollary 2 — Trust accumulation is prohibited

Observing correct outputs on inputs $x_1, \ldots, x_k$ provides no guarantee about $P(\varepsilon = 1)$ on a new input $x_{k+1} \notin \{x_1, \ldots, x_k\}$. The operative form's bound holds unconditionally over the distribution.

Practical implication: A system must not relax its verification layer after observing a run of correct outputs.

8.3 Corollary 3 — Verification scope must match error taxonomy

A verifier that checks only JSON schema conformance (`ERR_SCHEMA`) does not cover hallucination (`ERR_HALLUCINATION`) or sycophantic drift (`ERR_SYCOPHANCY`). Partial verification is better than none, but must be scoped honestly.

Practical implication: Verification claims must be scoped and documented, not asserted globally.

8.4 Corollary 4 — Silent acceptance is an architectural defect

Any production system that passes LLM output directly to downstream consumers without a declared verification boundary has an architectural omission. This defect exists regardless of observed output quality and regardless of model capability.

Practical implication: Code review should treat absence of an output validation layer as a blocking defect, equivalent to missing authentication on a sensitive endpoint.

8.5 Corollary 5 — Capability growth shifts the verification problem, not away from it

As model capability $C(M)$ increases, two things happen simultaneously:

1. **Low-stakes error classes become easier to detect.** `ERR_OMISSION`, `ERR_SCHEMA`, `ERR_TRUNCATION`, and `ERR_INSTRUCTION` failures tend to be structurally obvious. More capable models make these errors less frequently.

2. **High-stakes error classes become harder to detect.** `ERR_HALLUCINATION`, `ERR_SYCOPHANCY`, `ERR_SEMANTIC`, `ERR_CALIBRATION`, and `ERR_REASONING` failures become *more plausible* as model capability grows. A less capable model hallucinating a citation produces a nonsensical author name; a more capable model produces a real author name, a real journal, a plausible DOI, and an abstract that reads like the paper it should be. Detection now requires independently retrieving and reading the source.

Hypothesis (Capability-Detection Asymmetry): *[Labeled hypothesis — not derived formally. Both $C(M)$ and $D_c(M)$ require operational definitions before this can be tested empirically.]*

$$\frac{\partial D_c}{\partial C} \leq 0 \quad \text{for } c \in \{\texttt{ERR_OMISSION}, \texttt{ERR_SCHEMA}, \texttt{ERR_TRUNCATION}, \texttt{ERR_INSTRUCTION}\} \tag{6}$$

$$\frac{\partial D_c}{\partial C} > 0 \quad \text{for } c \in \{\texttt{ERR_HALLUCINATION}, \texttt{ERR_SEMANTIC}, \texttt{ERR_SYCOPHANCY}, \texttt{ERR_CALIBRATION}, \texttt{ERR_REASONIN} \tag{7}$$

Practical implication: Verification system upgrades are not optional post-deployment maintenance — they are a precondition for deploying a more capable model. Treating the verifier as a stable component while upgrading the model is an architectural regression.

9 Practitioner Artifacts

Copy-paste artifacts for enforcing PALS's Law at the function level (§9.1–9.3) and at the project level (§9.4).

9.1 Full Contract Block

For any function that invokes an LLM:

TypeScript contract block

```
/**
 * ARCHITECTURAL CONTRACT -- PALS's LAW
 *
 * Principle authored by: Pedro Anisio de Luna e Silva
 *
 * MODEL_VERSION: <model identifier and version>
 * PALS_LAW_VERSION: 1.5.4
 *
 * INVARIANT (operative form):
 *   E[e(M(x), x)] >= d > 0
 *
 * ERROR CLASSES NOT COVERED BY THIS CALLER'S VERIFIER:
 *   [ ] ERR_HALLUCINATION
 *   [ ] ERR_OMISSION
 *   [ ] ERR_SCHEMA
 *   [ ] ERR_TRUNCATION
 *   [ ] ERR_SYCOPHANCY
 *   [ ] ERR_INSTRUCTION
 *   [ ] ERR_CALIBRATION
 *   [ ] ERR_SEMANTIC
 *   [ ] ERR_REASONING
 *
 * Fill in the checklist above. Unchecked boxes are
```

```
 * known, accepted risks. Leaving all boxes unchecked
 * with no mitigation note is a blocking defect.
 */
```

9.2 Short-Form (headers, PR descriptions, commit messages)

```
ARCHITECTURAL REQUIREMENT (PALS's LAW):
LLM error rates are non-negligible across realistic
deployments. Absence of output verification is a design
defect, not a runtime bug. All LLM output must be treated
as untrusted and validated explicitly.
```

9.3 Inline Banner (single-line, for code comments)

```
// PALS's LAW: LLM output is untrusted by default.
// Verify before use.
```

9.4 CLAUDE.md Integration Block

Paste verbatim into the `LLM Output Verification` section of any `CLAUDE.md`:

```
## LLM Output Verification -- Architectural Requirement
## (PALS's LAW)

**Principle authored by:** Pedro Anisio de Luna e Silva
**PALS_LAW_VERSION:** 1.5.4

For any model M and realistic task distribution D:

    E[e(M(x), x)] >= d > 0

Every pipeline, agent, or workflow that accepts LLM
output MUST treat that output as untrusted, incomplete,
and unverified by default. Verification is not optional
post-processing -- it is a first-class design concern.

> Absence of a verification layer is a design defect,
> regardless of how correct the LLM output appears to be.
```

End of document — PALS's LAW v1.5.4